中國人民大學
RENMIN UNIVERSITY OF CHINA

单链表

插入

① 按位序 { 带头结点 / 不带头结点 → 特别考虑插入第一位 } O(1).

② 指定结点 { 前插 { ① 找前驱 ② (前驱不好找啊!) 将s插在*p后. 互换sp内容 } / 后插 → 常规操作 }

删除

① 按位序 { 常规 / 后继随对于尾删, 删除等后继 ⇒ O(1) }

② 按值

③ 指定结点 → 有坑: 当指定最后一个结点时.
如删除最后一个 P->next = NULL. 会返回 false.
只能从头遍历找前驱

查找

① 按位
② 按值
③ 求len.
} O(n) [遍历]

建立: while 插入 → $O(n^2)$ ⇒ 引入尾结点 → $O(n)$ (n个O(1))

初始化时: L->next = NULL 最好等上

① 尾插法
② 头插法 → 逆置

中国人民大学
RENMIN UNIVERSITY OF CHINA

队列

1. 普通队列

判空：front == rear

·不能 Q.rear == MaxSize （尾溢出头不满）"假溢出"

2. 循环队列

关键：Q.front = (Q.front+1) % MaxSize

rear.len.同理.

判空：

·不能 front == rear （满也是）

①牺牲一个单元 (Q.rear+1) % MaxSize == Q.front % MaxSize

②增设 size

③增设 tag （初始化 tag=0）

△ a. 初始化、入队、出队 循环加1%

b. 判空、判满

c. 长度

②③分别多定义一个 size 和 tag （初始化所段）

北京大学
PEKING UNIVERSITY

1. Hash表填装因子 $\alpha = \frac{\text{记录数}}{\text{Hash表长}}$ （装满程度）

2. ASL ——量化了查找效率

(一) Hash表

0	1	2	3	4	5	6	7	8	9
	8	9	31	11	5	27	15	16	

① 成功

8	9	31	11	5	27	15	16
1	1	1	2	1	1	7	7

$ASL_{成功} = \frac{1\times6+2\times7}{8} = 2.5$

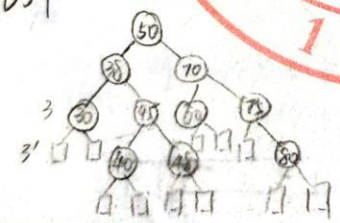
② 失败

若 H(key) = key % 7

0	1	2	3	4	5	6
1	9	8	7	6	5	4

$ASL_{失败} = \frac{1+9+\dots+4}{7} = \frac{40}{7}$

(二) BST



① 成功

$ASL_{成功} = \frac{1\times1+2\times2+4\times3+3\times4}{10}$

② 失败

因为没找到 $ASL_{失败} = \frac{5\times3+6\times4}{11}$

(十)

北京大学 PEKING UNIVERSITY

散列表（Hash）

1、散列函数：建立"关键字→存储地址"的映射关系

↓

散列表

理想情况下，散列表查找为O(1)

2、散列函数的构造

① <重>除留余数法 $H(key)=key\%p$ ✓质数

② 直接定址法 $H(key)=a*key+b$

（线性函数，不会产生冲突，但可能造成存储空间浪费较多）

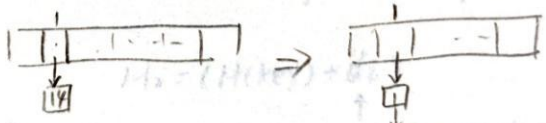
③ 数字分析法：选分布均匀的一部分，eg. 手机尾号

④ 平方取中法：选关键字平方值的中间几位作为散列地址

3、解决冲突

(1) 拉链法

① (头插) 插入操作 eg: 14%13=1, 1%13=1



② 查找操作：先算hash地址，找该地址下的链表是否有目标元素（为空位置则为空）

③ 删除操作：查找到删除，只一个删除后置空

（若链表保持有序，可略微提高"查找"效率）

北京大学
PEKING UNIVERSITY

(2) 开放定址法 ①线性探测法

$H_i = (H_{i-1} + d) \% p$

(往后面查找到空闲位置)

不能简单删除，但可以逻辑删除（如标记 "delete"）.

物理，

优点：只要表不满，总能找到空位

缺点：①$h(k_1) = i$，$h(k_2) = j$，k_1 可能占据 k_2 位置，性能下降

②表较满时，性能通常很差

复杂性：初始化：$O(m)$

搜索、插入最差情况：$O(n)$

② 平方探测法 ③伪随机序列

$H_i = (H(key) + d_i) \% p$ $(d_i = 0^2, 1^2, 2^2, 3^2, \ldots)$

(3) 双散列法

设计双 hash 函数

eg. $H(key) = key \% 13$

$hash_2(key) = 13 - (key \% 13)$

$H_i = (H(key) + d_i)$

$d_i = i \times hash_2(key)$

北京大学
PEKING UNIVERSITY

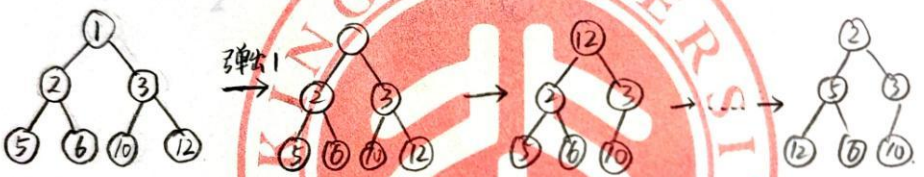
(十二)
优先队列、堆、哈夫曼树

堆：1、下滤：与子结点比较 → 自下而上建堆
2、上滤：与父结点比较 → 自上而下建堆

$O(\log_2 n)$　　$O(n)$

应用：优先队列（小根堆）

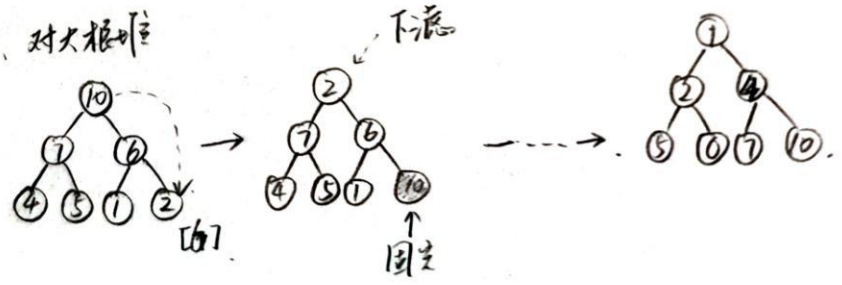
① 弹出最小元素（堆顶），之后把最后一个置顶，做一遍下滤



② 如果重复上述弹出操作，我们会得到 1 2 3 5 6 10 12
这就是"堆排序"

但为了降低空间复杂度，我们往往把弹出元素放在"空缺的地方"（交换）之后一维数组就是有序的。

例、对大根堆



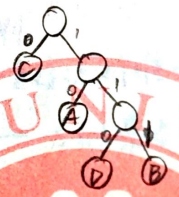
哈夫曼树

1、$WPL = \sum_{i=1}^{n} w_i l_i$

哈夫曼树要求WPL最小

2、哈夫曼编码

选项	频数
A	10
B	8
C	80
D	2



C	0
A	10
D	110
B	111

PEKING UNIVERSITY 1898

(十四).

BST、AVL

一、BST (二叉搜索/排序树)

① 插入 → 构建

(递归方法) 最坏时间复杂度: O(h).

② 删除p

(1) 删除叶结点p —直接删

(2) p有且只有一个非空子树—直接让这个唯一孩子代替他

(3) p既有左子树又有右子树— 左树最右或右树最左代替之

③ 查找

ASL = $\frac{\sum n_i \cdot l_i}{N}$ ← 评估指标

因此 BST 高度要尽量小 → ASL 小 → 接近 O(log n)

(平衡二叉树)

AVL

北京大學
PEKING UNIVERSITY

二、AVL（首先是BST）

1、插入（按BST插，之后AVL调平）$|\Delta h| \leq 1$

{ 左旋：逆时针
  右旋：顺时针 左逆右顺

LL型：右旋

RR型：左旋

LR型：R的先左旋，L的右旋
↑主 ↑孩

RL型：L的先右旋，R的左旋

2、删除

① 用BST方法删除

② 一直向上找不平衡结点

③ 找到最小不平衡树下最高的儿子、孙子

④ 根据孙子位置，调节

孙子：LL
RR
LR
RL
（同上）

⑤ 如果不平衡性沿上传递，重复以上过程

北京大学 PEKING UNIVERSITY

三、红黑树（首先是BST）

1、适用于频繁插入删除的场景（相较于AVL）

左根右(BST)、根叶黑

不连红 黑路同

任意根结点出发到叶结点，路结点黑结点相同

黑结点数量称"黑高"

2、性质

① 从根结点到叶结点的最小路径不大于最短路径×2

∵ 红≤黑-1

∴ 红+黑≤2×黑-1<2×黑

② 有n个内部结点的红黑树高度 $h \le 2\log_2(n+1)$

查找时间复杂度 $O(\log_2 n)$

3、操作

① 查找：同BST

② 插入：(1) BST插入

(2) 新结点是根 —— 染黑

新结点非根 —— 染红

若满足红黑树，则结束

若不满足，则重调

判断"不连红"即可

- 黑叔：无权 —— 左转+染色
  - LL：
  - RR：
  - LR：
  - RL：

  先AVL方法 → 原本父爷反色 / 原本小爷反色
- 红叔：染色+变新 —— 叔父爷反色、爷变新结点

北京大學
PEKING UNIVERSITY

关于"Bh(黑高)"的推论：

(1) 高为h的红黑树，内部结点最少 $2^{\frac{h}{2}}-1$ （无红）

(2) 若总高为h，则 $Bh \geq \frac{h}{2}$，因此 $n \geq 2^{\frac{h}{2}}-1$

$\therefore \frac{h}{2} \leq \log_2^{(n+1)}$

$h \leq 2\log_2^{(n+1)}$

③ 删除

时间复杂度 $O(\log_2 n)$

(方式同 BST)

PEKING UNIVERSITY 1898

(对非根)
子结点数量 $\lceil \frac{m}{2} \rceil \le c \le m$ → 分支
关键字数量 $\lceil \frac{m}{2} \rceil - 1 \le n \le m-1$ → 元素

北京大学
PEKING UNIVERSITY

四 B树 （多路平衡查找树） → 每一块都是在划分数字范围

1. 平衡：所以失败结点都在同一层
   有序
   多路：（m阶B树）
   - 最多：m个分支、m-1个元素
   - 最少：
     - 根结点：2个分支、1个元素
     - 其他：$\lceil m/2 \rceil$ 个分支，$(\lceil m/2 \rceil - 1)$ 个元素

2. 操作
   ① 查找
   ② 插入 先查找放入
      - 若未上溢出：不变
      - 若上溢出：中间元素 $\lceil \frac{m}{2} \rceil$ 上溢出，两边分裂
   ③ 删除：先查找
      - 非叶结点：用直接前驱或者按后继替换
      - 叶结点：直接删除

      ↓

      - 若未下溢出：不变（左右兄弟）
      - 若下溢出：
        - 兄弟够借：从下兄弟
        - 不够借：合并（父亲合并）

（具体自己理解就行）

不存在连续
复用操作 ← 符合直觉

五、B+树

B

所有结点为键字都有直接指向对应记录的指针

m个结点 m-1关键字

随机查找，但不能顺序

B+

B+树非叶结点只作索引

m个结点、m个关键字

都有序

PEKING UNIVERSITY 1898

北京大学
PEKING UNIVERSITY

(十六)
图(一)

一、图的存储

1. 邻接矩阵 (顺序)

{ 无向图
  有向图 (离散一样)

(不相连为0)

↓

带权图 (不相连为∞)

↓

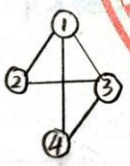
#define INFINITY 最大int值

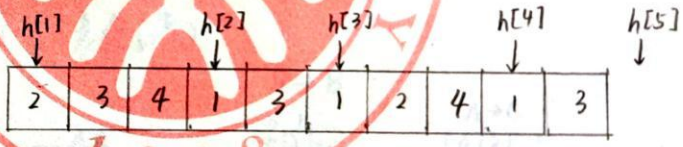
A
A^2
A^3
⋮
A^{n-1}
图论

2. 邻接压缩表 (顺序)

防止空间浪费 (处理过于稀疏的矩阵)

例:





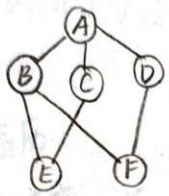
h=[0, 3, 5, 8, 10]

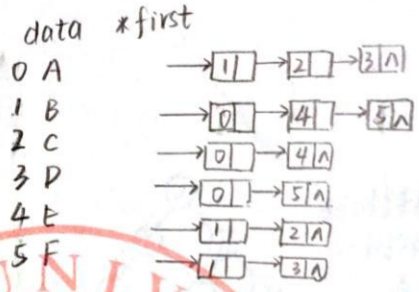
(有向同理)

北京大学
PEKING UNIVERSITY

3. 邻接链表（链表存储）

例（无向）





边结点数量为 2|E|

空间复杂度 O(|V|+2|E|)

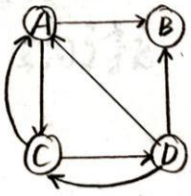
（有向）

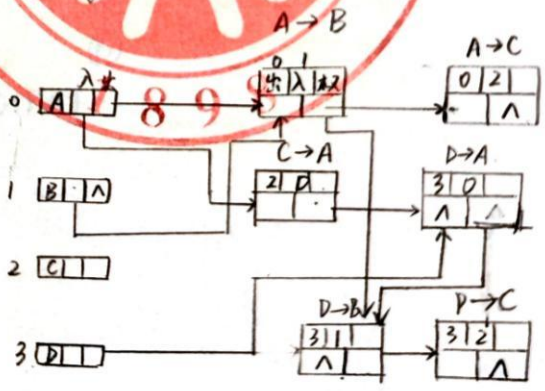
边结点数量为 |E|

空间复杂度 O(|V|+|E|)

4. 十字链表（链表存储）

例：





（按元素明细录行）

按照顺序为：

01 02 03 ...

31 32 ...

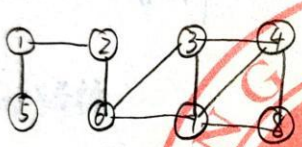
北京大学
PEKING UNIVERSITY

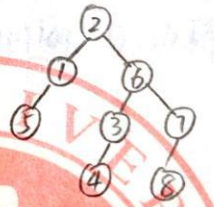
二、图的基本操作

知道时间复杂度

三、图的遍历

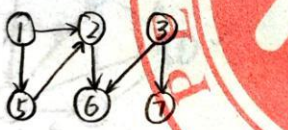
1、图的广度优先（BFS）





时间复杂度：
访问邻接矩阵 $O(|V|^2)$
邻接表 $O(|V|+|E|)$

2、图的深度优先（DFS）





时间复杂度

同上（算法结构相关）

最坏递归深度 $O(|V|)$

（空间复杂度）

PEKING UNIVERSITY 1898

(十六)
图(二)

四、最小生成树

1、Prim

$O(|V|^2)$：边稠密

圈圈每次选最优的

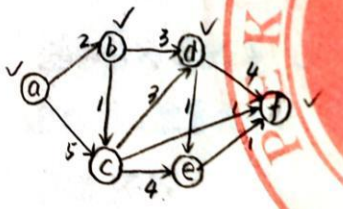
2、Kruskal

$O(|E|\log_2|E|)$：边稀疏

先挑短边开行

五、最短路径

1、Dijkstra算法



第i次	2	3	4	5
b	2 (a,b)			
c	5 (a,c)	3 (a,b,c)		
d	∞	5 (a,b,d)	5 (a,b,d)	
e	∞	∞	7 (a,b,c,e)	7 (a,b,d,e)
f	∞	∞	4 (a,b,c,f)	
终点集	{a,b}	{a,b,c}	{a,b,c,f}	{a,b,c,f,d,e}

{a,b,c,
f,d

BFS —带权→ Dijkstra —负权→ Floyd

无权

(带负权回路不行)

北京大学
PEKING UNIVERSITY

2、Floyd 算法

$D^{(-1)} \to D^{(0)} \to D^{(1)} \to D^{(2)} \cdots$

$Path^{(-1)} \to Path^{(0)} \to Path^{(1)} \to Path^{(2)} \cdots$

D表出权　Path表出路径

（根据终点前驱查找）

六、拓扑排序

AOV网：有向无环，顶点表示活动

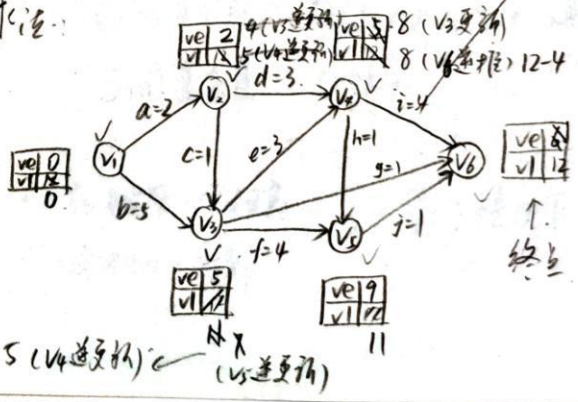
选入度为0的点，选中后删除，以此类推

逆拓扑：出度为0

七、关键路径

AOE网：顶点代表事件，边代表活动（发生里行…）

求法：



逆拓扑"出度为0"

更新"逆拓扑跟前驱"的等为ve

拓扑求ve

逆拓扑求vl

e=L的边构成关键路径

边：(发出点)

	a	L
a	Vi	Vi+1-W
⋮		
j		

北京大学
PEKING UNIVERSITY

一、插入排序 ← 从后往前 < 最好 $O(n)$ / 最差 $O(n^2)$ => 平均 $O(n^2)$

折半插入

a[0]存储待查元素，之后 high, low, mid 三指针操作

直至 low > high，右移覆盖，a[0]放入

[low, i-1]

- 当 A[mid] == A[0] = true 时，继续 low = mid+1 就好
- 比较次数变了，移动次数没变，仍然 $O(n^2)$

二、希尔排序

适用：基本有序，小数据量（由于d→随机访问→仅用于顺序表）

最坏 $O(n^2)$ → n在某范围：$O(n^{1.3})$ → 一般认为优于 $O(n^2)$

三、快速排序（☆代码）

时间复杂度：$O(k \cdot n)$　　k：递归深度 $\in [\log_2 n + 1, n]$ ⇓ 完成全序 $O(n^2)$（归并）

空间复杂度：$O(k)$

- 一趟排序：全扫完
- 一次划分：局部

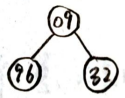
平均情况下，$O(n \log_2 n)$.

简单选排 | 时 $O(n^2)$
| 空 $O(1)$

北京大学
PEKING UNIVERSITY

四、堆排序

大根堆不断"弹出"，考虑到空间，往往把弹出元素放在"它该放的地方"

- 

09下坠时，先对比96和32谁大，再与09比较并下坠

  同比，结点下坠最多 $(h-1)$ 层，对比 $[2(h-1)]$ 次

  (第1层)

- 建堆 时间：$O(n)$

  每趟排序 时间：$O(\log_2 n)$ // $O(n\log_2 n)+O(n)$

  $(n-1)$ 趟/总：时间 $O(n\log_2 n)$.

- 空间 $O(1)$.

北京大學
PEKING UNIVERSITY

五、归并排序

"先打散 再合并" (递归) —— 二路归并

归并层数（指数）$k=\log_2 n$

每趟时间 $O(n)$

归并总时间 $O(n\log_2 n)$

空间 $O(n)=O(n)+O(\log_2 n)$

辅助数组 递归

PEKING UNIVERSITY 1898

北京大学
PEKING UNIVERSITY

六、箱排序

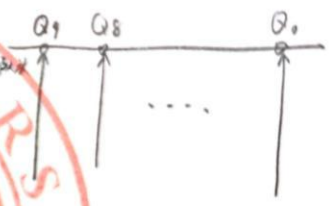
尽量均匀分，内部排完 排桶序

时间：$O(n+k)$
（k：桶）

七、基数排序

使用队列

按个、十、百……依次排（借队列）



时间 $O(d*(n+r))$
（d：位；r：基数[0~9]）
（辅助队列数量）

八、计数排序（给出取值范围和数量关系）

Step 1、（待排元素 [0,k)），建立长为k的辅助数组，统计出现次数

Step 2、统计 C[i]，=i的元素个数

Step 3、辅助数组实现排序，放一次C[i]--

空间 $O(n+k)$ ⇒ 和k的数量级相关

时间 $O(n+k)$
（n：输出数组；k：辅助数组C）

北京大學
PEKING UNIVERSITY

分析与比较

	最好时间复杂度	最坏
冒泡	n^2	n^2
计数	n^2	n^2
插入	n^2	n^2
选择	n^2	n^2
堆	$n\log_2 n$	$n\log_2 n$
归并	$n\log_2 n$	$n\log_2 n$
快速	n^2	$n\log_2 n$
	初始逆序	

简单排序
适用于待排序列基本有序或n较小

结论

1. 平均来看，快排最优，
但初始有序会降为$O(n^2)$，可采用三平均选中轴法 (pivit)
tt较 low、mid、high 找出 pivit

2. 稳定性：相同元素相对位置是否改变

稳定：插入、冒泡、归并、基数

不稳定：简单选择、希尔、快速、堆